

Task 1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```

shreyas@Grigori:~$ nano filecopy.c
shreyas@Grigori:~$ gcc filecopy.c -o filecopy
shreyas@Grigori:~$ echo "This is a test file for low-level Linux I/O." > source.txt
shreyas@Grigori:~$ ./filecopy source.txt destination.txt
File copied successfully.
shreyas@Grigori:~$ cat destination.txt
This is a test file for low-level Linux I/O.
shreyas@Grigori:~$ nano filecopy_stdio.c
shreyas@Grigori:~$ gcc filecopy_stdio.c -o filecopy_stdio
shreyas@Grigori:~$ ./filecopy_stdio source.txt destination_stdio.txt
File copied successfully.
shreyas@Grigori:~$ cat destination_stdio.txt
This is a test file for low-level Linux I/O.
shreyas@Grigori:~$ |

```

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    src = open(argv[1], O_RDONLY);

    if (src < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0)
    {
        perror("Error opening destination file");
        close(src);
        return 1;
    }

    while ((bytesRead = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        while ((bytesRead = read(src, buffer, BUFFER_SIZE)) > 0)
        {
            bytesWritten = write(dest, buffer, bytesRead);

            if (bytesWritten != bytesRead)
            {
                perror("Error writing file");
                close(src);
                close(dest);
                return 1;
            }
        }

        if (bytesRead < 0)
            perror("Error reading file");

        close(src);
        close(dest);

        printf("File copied successfully.\n");

        return 0;
    }
}

```

Task 2: Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection

```
shreyas@Grigori:~$ nano redirect.c
shreyas@Grigori:~$ gcc redirect.c -o redirect
shreyas@Grigori:~$ ./redirect
shreyas@Grigori:~$ cat output.txt
This output has been redirected to output.txt
Standard output is now connected to the file.
shreyas@Grigori:~$ echo "Hello from shell" > shell_output.txt
shreyas@Grigori:~$ cat shell_output.txt
Hello from shell
shreyas@Grigori:~$ echo "Linux input redirection test" > input.txt
shreyas@Grigori:~$ cat < input.txt
Linux input redirection test
shreyas@Grigori:~$ ls > directory.txt
shreyas@Grigori:~$ cat directory.txt
DIRECTORY...
DSE0BD
File1.txt
File2.txt
WLH-AI-and-DS-2026-27-Team-9-Memory-Mapped-File-Management-System
Makefile
OSSP
Week4
[OSLAB]...
command_executor
command_executor.c
command_runner
command_runner.c
copy_on_write
data
destination.txt
destination_stdio.txt
directory.txt
docs
fifo_client_server
file1.c
file1.txt
filecopy
filecopy.c
filecopy_atudio.c
```

```
hello.c
input.txt
linux-mmap-filesystem
ls_grep_pipe
memory_allocation
memory_analysis
memory_layout
mmap-project-organized
mmap-project-organized.zip
mmap-project-organized.zip:Zone.Identifier
name.txt
output.txt
process_states
process_states.c
producer_consumer_pipe
random.txt
redirect
redirect.c
reports
results
shell_output.txt
signal_handling
skill_week4
source.txt
src
task1
task1.c
task2
task2.c
task3
task3.c
task4
task4.c
week4_t1
week4_t1.c
week4_t2
week4_t2.c
shreyas@Grigori:~$ echo "Second line" >> directory.txt
shreyas@Grigori:~$ cat directory.txt
DIRECTORY...
DSEDBD
```

```
File2.txt
KUH-AI-and-OS-2026-27-Team-9-Memory-Mapped-File-Management-System
Makefile
OSSP
Week4
[OSLAB]...
command_executor
command_executor.c
command_runner
command_runner.c
copy_on_write
data
destination.txt
destination_stdio.txt
directory.txt
docs
fifo_client_server
file1.c
file1.txt
filecopy
filecopy.c
filecopy_atudio.c
filecopy_stdio.c
filecopy_stdio.c
filecopy_stdio.c
fork_example.c
hello.c
input.txt
linux-mmap-filesystem
ls_grep_pipe
memory_allocation
memory_analysis
memory_layout
mmap-project-organized
mmap-project-organized.zip
mmap-project-organized.zip:Zone.Identifier
name.txt
output.txt
process_states
process_states.c
producer_consumer_pipe
```

```
hello.c
input.txt
linux-mmap-filesystem
ls_grep_pipe
memory_allocation
memory_analysis
memory_layout
mmap-project-organized
mmap-project-organized.zip
mmap-project-organized.zip:Zone.Identifier
name.txt
output.txt
process_states
process_states.c
producer_consumer_pipe
random.txt
redirect
redirect.c
reports
results
shell_output.txt
signal_handling
skill_week4
source.txt
src
task1
task1.c
task2
task2.c
task3
task3.c
task4
task4.c
week4_t1
week4_t1.c
week4_t2
week4_t2.c
Second line
shreyas@Grigori:~$ |
```

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;

    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("This output has been redirected to output.txt\n");
    printf("Standard output is now connected to the file.\n");

    return 0;
}
```